COMS 4995-HW3

IAN CHING YI WANG

March 2026

The code and implementation for this project are available at:
`https://github.com/IanWang2002/COMS-4995-Homework-3.git`.

1 Introduction

The Transformer architecture has become a foundational model in modern natural language processing and large language models due to its ability to model long-range dependencies through self-attention. Unlike recurrent architectures, Transformers process tokens in parallel while using attention mechanisms to determine which previous tokens are most relevant for predicting the next one. This makes them both conceptually important and practically effective for sequence modeling tasks.

In this project, we implement a lightweight Transformer language model from scratch in PyTorch for next-token prediction on the Tiny Shakespeare corpus. The goal is not to build a large-scale language model, but rather to understand the core components that make the Transformer effective. Our implementation includes subword tokenization using Byte Pair Encoding (BPE), token embeddings with positional encodings, masked self-attention, residual connections, RMSNorm, and a feed-forward network. The model is trained to predict the next token in a sequence, and its performance is evaluated using validation loss and perplexity.

Beyond predictive performance, this project also examines the internal behavior of the model through visualization. In particular, attention heatmaps provide insight into how the Transformer distributes focus across previous tokens when making predictions. By combining implementation, training, and interpretation, this project offers a compact but informative study of how Transformer-based language models operate on sequential text data.

2 Methods

2.1 Data & Preprocessing

We use the Tiny Shakespeare corpus for the next-token prediction task. This dataset is a compact benchmark for language modeling and is well suited for studying Transformer behavior in a computationally lightweight setting. The raw text is loaded as a continuous text stream and then converted into token sequences for training.

To satisfy the requirement of subword tokenization, we use **Byte Pair Encoding (BPE)** to map text into integer token IDs. The vocabulary size is limited to 500 in order to keep the model lightweight while still capturing meaningful subword units such as common prefixes, suffixes, and frequent word fragments. This provides a more informative representation than character-level tokenization while remaining computationally efficient.

After tokenization, the corpus is split into overlapping fixed-length sequences of 50 tokens. For each example, the input consists of 50 consecutive tokens, and the target sequence is formed by shifting the input by one position, so that the model learns to predict the next token at every time step. A stride of 1 is used when forming sequences, which maximizes the use of the available text.

This preprocessing produces 448,079 input-target pairs in total. The data is then divided into **80% training data and 20% validation data**, with no separate test set. We preserve the sequential ordering of the corpus when splitting, resulting in 358,463 training sequences and 89,616 validation sequences. To ensure reproducibility, random seeds are fixed across Python, NumPy, and PyTorch before training.

2.2 Model Architecture

We implement a minimal Transformer-based language model in PyTorch for next-token prediction. The model is intentionally kept lightweight, using two Transformer blocks with hidden dimension 64, which satisfies the assignment recommendation of using a small architecture for efficient experimentation. Given an input sequence of token IDs, the model first maps each token to a dense vector representation using an embedding layer. Sinusoidal positional encodings are then added to these embeddings so that the model can incorporate information about token order.

The core component of the model is the self-attention module. For each input sequence, the model computes query, key, and value representations through learned linear projections. Attention scores are obtained by taking the scaled dot product between queries and keys, followed by a softmax operation to produce attention weights. These weights determine how strongly each token attends to previous tokens when forming its contextualized representation.

Because the task is next-token prediction, a causal mask is applied to the attention matrix to prevent tokens from attending to future positions. This ensures that the prediction at each time step depends only on the current token and earlier context, rather than leaking information from later tokens in the sequence. The resulting attention output is then passed through a linear projection.

Each Transformer block also contains a feed-forward network composed of two linear layers with a ReLU activation in between. Residual connections are applied around both the self-attention sublayer and the feed-forward sublayer to improve optimization and preserve information flow. In addition, RMSNorm is used before each sublayer to stabilize training. A dropout layer is included in the residual pathways to reduce overfitting.

After the stacked Transformer blocks, a final RMSNorm layer is applied, followed by a linear output layer that maps hidden representations to vocabulary-sized logits. These logits are used to predict the next token at each position in the sequence. The model is trained using **cross-entropy loss**, which compares the predicted token distribution against the shifted target sequence.

2.3 Training Setup

The model is trained for the next-token prediction task using cross-entropy loss. For each input sequence, the target sequence is the same token sequence shifted by one position, so the model learns to predict the next token at every time step. The output logits are reshaped across batch and sequence dimensions before computing the loss against the target token IDs.

We optimize the model using the AdamW optimizer with a learning rate of 1×10^{-3}. Training is performed with a **batch size of 32**. We compared learning rates of 1×10^{-4}, 3×10^{-4}, and 1×10^{-3} based on validation loss during the first three epochs, and found that 1×10^{-3} achieved the best early validation performance. To further improve generalization, dropout with rate 0.1 is applied within each Transformer block, and early stopping with patience 2 is used during training.

Model performance is monitored on the validation set after each epoch. During validation, gradients are disabled using `torch.no_grad()` to reduce memory usage and computation. We also apply early stopping with patience 2 based on validation loss, saving the model parameters whenever validation performance improves. This allows us to retain the best-performing model while reducing unnecessary overtraining.

For reproducibility, random seeds are fixed across Python, NumPy, and PyTorch before training. All experiments are implemented entirely in PyTorch.

3 Results

We evaluate the Tiny Transformer using validation loss, perplexity, attention visualization, and qualitative text generation. Overall, the model learns meaningful sequential patterns from the Tiny Shakespeare corpus while remaining computationally lightweight.

3.1 Learning Rate Selection

To select the optimizer learning rate, we compared three values: 1×10^{-4}, 3×10^{-4}, and 1×10^{-3}. Figure 1 shows the validation loss across the first three epochs for each setting. Among the tested values, 1×10^{-3} consistently achieved the lowest validation loss at every epoch, indicating the fastest and most effective convergence in the early stage of training. Therefore, 1×10^{-3} was selected for the final training run.

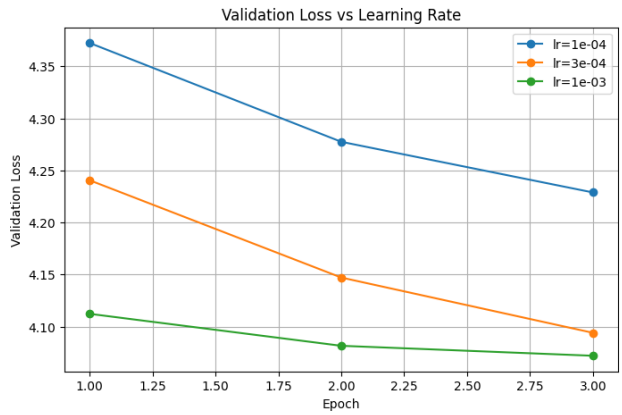

Figure 1: Validation loss across epochs for different learning rates. The learning rate 1×10^{-3} consistently achieves the lowest validation loss, so it was selected for final training.

3.2 Training and Validation Loss

Figure 2 shows the training and validation loss over epochs. The training loss decreases steadily, indicating that the model successfully learns token-level patterns from the training data. The validation loss improves in the early stage of training and then gradually plateaus, suggesting mild overfitting as training continues. The best validation loss achieved is approximately 4.0188 at epoch 8, while the training loss continues decreasing beyond that point. Early stopping was triggered at epoch 10, which helps limit overfitting by retaining the model at its best validation performance rather than continuing to train after validation loss has plateaued. This behavior is consistent with a lightweight Transformer trained on a relatively small corpus, where the model quickly captures common local patterns but has limited capacity for further generalization.

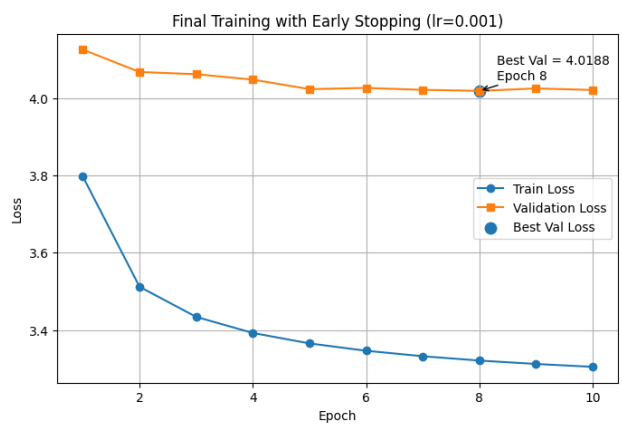

Figure 2: Training and validation loss over epochs. Training loss decreases steadily, while validation loss improves early and then plateaus, indicating mild overfitting after initial convergence.

3.3 Perplexity

We report perplexity on the validation set as the main evaluation metric, computed as the exponential of the validation cross-entropy loss:

$$\text{PPL} = \exp(\mathcal{L}_{\text{val}})$$

Perplexity measures how uncertain the model is when predicting the next token. A lower perplexity indicates better predictive performance, since it means the model assigns higher probability to the correct next token on average. In language modeling, perplexity can be interpreted as the effective number of plausible token choices the model is considering at each step.

Using the best validation loss of 4.0188, **the model achieves a validation perplexity of approximately 55.64.** This is a reasonable result given the compact architecture, limited vocabulary size, and short training schedule. It suggests that the model has learned meaningful sequential and stylistic patterns from the corpus, although there is still considerable uncertainty in next-token prediction, which is also reflected in the limited coherence of the generated text.

3.4 Attention Visualization

To better understand model behavior, we visualize attention for individual sequences, across both Transformer blocks, and across training epochs. Figure 3 shows a representative attention heatmap for one validation sequence. The clear lower-triangular structure confirms that the causal mask is functioning correctly, while the strong concentration near the diagonal indicates that the model relies primarily on nearby context for next-token prediction.

Figure 4 extends this analysis to two validation sequences across both Transformer blocks. In both cases, Block 1 exhibits strongly local attention concentrated near the diagonal, while Block 2 shows slightly more selective and structured patterns. This suggests that deeper layers refine the contextual representations formed by earlier layers rather than simply repeating the same local pattern. Some off-diagonal attention is also visible, indicating limited use of earlier context beyond the immediately preceding token, although long-range dependency modeling remains constrained in this lightweight setting.

Figure 5 compares attention for the same sequence across training epochs. Early in training, attention is more diffuse, while later epochs show sharper concentration and a clearer diagonal structure. This suggests that the model gradually learns more stable token-to-token relationships as optimization progresses.

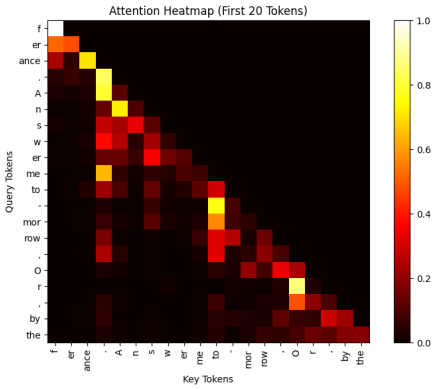

Figure 3: Representative attention heatmap for a validation sequence. The lower-triangular structure confirms correct causal masking, and the strong diagonal pattern indicates that the model primarily attends to nearby preceding tokens.

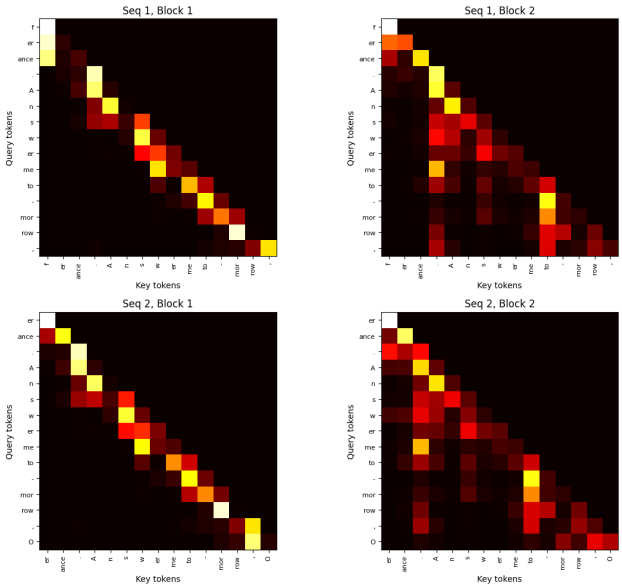

Figure 4: Attention heatmaps for two validation sequences across both Transformer blocks. Each row corresponds to a query token and each column corresponds to a key token. Block 1 shows strongly local attention near the diagonal, while Block 2 appears more selective and structured, suggesting a refined use of contextual information.

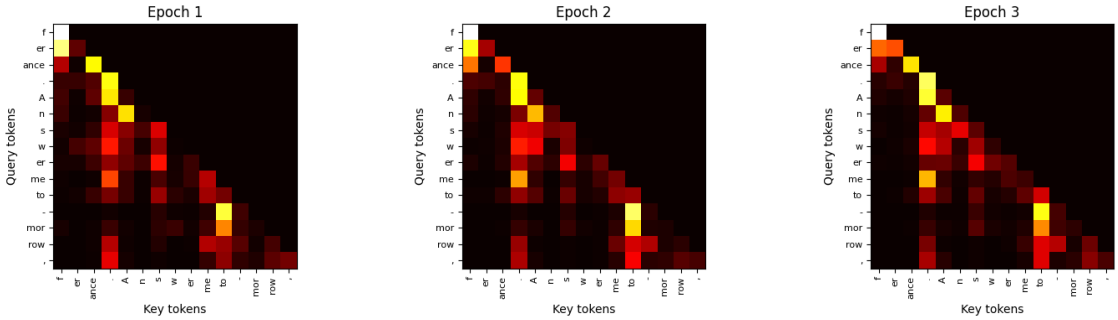

Figure 5: Attention heatmaps for the same validation sequence across training epochs. Early attention is more diffuse, while later epochs show sharper and more concentrated patterns near the diagonal, indicating that the model learns increasingly stable local token relationships during training.

3.5 Qualitative Generation

To qualitatively evaluate the model, we generated text from the prompt "To be or not to be." One sample output is shown below:

> To be or not to be g in as ion of the com m on A n ce ! w hy ' he hath d one . P a ul in the ac ter ' s word . W AR W IC K : U n mo ther ' s de ad he con j e

The generated text reflects partial learning of Shakespeare-like style, including fragments of dramatic phrasing, punctuation, and word patterns. However, the output is only locally coherent and contains visible subword fragmentation. This behavior is expected given the lightweight architecture, limited training time, and constrained BPE vocabulary size. Overall, the sample suggests that the model has learned some stylistic and syntactic structure, but still struggles with semantic consistency and fluent long-range generation.

4 Discussion and Conclusion

The lightweight Transformer learns meaningful local structure from the Tiny Shakespeare corpus. Training loss decreases steadily, while validation loss improves early and then plateaus, indicating mild overfitting. The attention heatmaps confirm correct causal masking through their lower-triangular structure and show that the model relies mainly on nearby context, with some limited longer-range dependencies.

The learning rate had a clear effect on performance. Among the tested values, 1×10^{-3} achieved the lowest validation loss and was selected for final training with early stopping. Positional encoding was also essential, since self-attention alone does not represent token order. Overall, the model captures some Shakespeare-like syntax and style, but generation remains only partially coherent because of the small architecture, short context length, and limited vocabulary.

Overall, this project shows that the main Transformer components—token embeddings, positional encoding, masked self-attention, residual connections, RMSNorm, and feed-forward layers which can be implemented from scratch to build a working language model. Although the model is limited in fluency and long-range consistency, it effectively demonstrates the core ideas behind Transformer-based sequence modeling. Future improvements could include using a larger vocabulary to reduce subword fragmentation, adopting multi-head attention to capture more diverse token relationships, and increasing the context length to better model longer-range dependencies.

Table 1 shows that larger learning rates led to better performance in this experiment. The smallest learning rate, 1×10^{-4}, converged slowly and produced the worst validation loss and perplexity, while 3×10^{-4} improved both metrics. The best results were obtained with 1×10^{-3}, which achieved the lowest validation loss and perplexity and was therefore selected for final training.

Learning Rate	Best Val Loss	Perplexity	Observation
1×10^{-4}	4.2288	68.63	Slowest convergence, highest validation loss
3×10^{-4}	4.0940	59.98	Improved convergence, better validation performance
1×10^{-3}	4.0188	55.64	Best validation performance, selected for final training

Table 1: Comparison of learning rates for the Tiny Transformer over the first three training epochs. The learning rate 1×10^{-3} achieved the lowest validation loss and perplexity, so it was selected for final training.

5 AI Tool Usage Disclosure

1. **List AI tools used:** ChatGPT (OpenAI)

2. **Contribution of AI Tools:** I used ChatGPT to support coding efficiency and documentation clarity. Specifically, it assisted in generating starter code templates, debugging syntax errors, refining evaluation pipelines, and improving LaTeX formatting for the final report. The tool was also used to help structure performance summaries and enhance the clarity of technical explanations.

3. **Author Contributions:** All data preprocessing decisions, modeling choices, hyperparameter selections, performance interpretation, and model reasoning were independently performed by the author. The complete model implementation, feature engineering decisions, and result analysis were developed through the author's own understanding of machine learning principles. Furthermore, all AI-generated suggestions were carefully reviewed, modified, and validated to ensure correctness and alignment with assignment requirements.